MIRAI Shield User Manual

A proposed guide for the IIoT PQC migration platform

Purpose
This manual explains how to prepare the upload dataset, interpret MIRAI Shield outputs, and use the policy engine and profile recommender for structured post-quantum cryptography migration planning in IIoT environments.

Authors

Borhan Rownak

Prof. Yuzo Taenaka

Prof. Youki Kadobayashi

Institution

Cyber Resilience Laboratory
Division of Information Science
Nara Institute of Science and Technology
Ikoma, Nara, Japan

Version 1.0

April 14, 2026

Contents

1 Introduction **2**
1.1 Overview . . . 2
1.2 Goal . . . 2

2 Platform Workflow **2**
2.1 Step-by-step flow . . . 3

3 System Modeling and Inventory Creation **3**
3.1 Purdue-layer modeling . . . 4
3.2 Data and Impact Inventory . . . 4
3.3 Cryptographic Inventory . . . 4

4 Dataset Preparation for Upload **4**
4.1 Important formatting rules . . . 4
4.2 Recommended required columns . . . 5
4.3 Unified PQC and Migration aware conduit inventory (UPMI) attribute guide . . . 5

5 How to Use the Platform **7**
5.1 Step 1: Upload the dataset . . . 7
5.2 Step 2: Run the Policy Engine . . . 7
5.3 Step 3: Run the Profile Recommender . . . 7

6 Interpreting the Output **8**
6.1 Algorithm rank . . . 8
6.2 Best algorithm . . . 8
6.3 Confidence score . . . 8
6.4 Migration priority . . . 8

7 Recommended Review Process **8**

8 Common Upload Mistakes **9**
8.1 Default Weights and User Customization . . . 9

9 Illustrative Case Study: Smart Grid UPMI Example **12**
9.1 Example Dataset Link . . . 12

10 Conclusion **12**

1. Introduction

1.1 Overview

MIRAI Shield is an IIoT-specific framework for post-quantum cryptography migration. Built on industry-oriented Purdue-layer system modeling, the framework provides a stepwise methodology for creating a unified cryptographic inventory of industrial assets and communication conduits. Based on this structured inventory, the policy engine evaluates each record to determine the required PQC security category, identify feasible protocol-update paths, and account for practical deployment constraints. The framework then produces ranked cryptographic recommendations, confidence scores, and migration priorities to support evidence-based migration planning.

1.2 Goal

The primary goal of MIRAI Shield is to support a smooth, practical, and understandable transition from classical cryptography to PQC in IIoT infrastructures. The spreadsheet-style input format and Purdue-layer modeling approach are intentionally designed to improve usability for industrial practitioners and to make migration planning transparent, structured, and repeatable.

2. Platform Workflow

MIRAI Shield follows the workflow shown in Figure 1. The process starts with system modeling and inventory creation, then consolidates the information into a Unified PQC- and Migration-Aware Conduit Inventory (UPMI). The policy engine determines the realistically deployable security category, after which the profile recommender ranks candidate algorithms and reports confidence and migration priority.

Figure 1: MIRAI Shield architecture for PQC migration.

2.1 Step-by-step flow

1. Model the target infrastructure and assign assets to Purdue layers and zones.
2. Create the **Data and Impact Inventory** and the **Cryptographic Inventory**.
3. Merge these inputs into the **UPMI** dataset.
4. Upload the dataset to the MIRAI Shield platform.
5. Run the **Policy Engine** to obtain the possible deployed PQC category for each conduit.
6. Run the **Profile Recommender** to obtain ranked algorithms, best candidates, confidence scores, and migration priorities.
7. Review the outputs and refine the migration plan through expert validation.

3. System Modeling and Inventory Creation

3.1 Purdue-layer modeling

Before preparing the dataset, practitioners should model the target system using Purdue layers and security zones. Each communication relationship should be represented as a conduit between a source zone and a destination zone. This makes it possible to connect operational context, data sensitivity, and deployment feasibility to the eventual PQC recommendation.

Table 1: IIoT Layers (Purdue Layer)

Level	**Description**	**When to require BOTH (auth + encryption)**
L1 – Field (sensors/actuators, IIoT nodes)	Sensor telemetry, actuator/drive commands, firmware updates; BLE/Zigbee/LoRa/WirelessHART; some wired fieldbuses	Not Applicable for this study
L2 – Cell/Area Control (PLCs, gateways, I/O nets)	PLC ↔ remote I/O, PLC ↔ gateway, HMI ↔ PLC; Modbus, DNP3, PROFINET, OPC UA	Conditionally needed both
L3 – Operations/SCADA (SCADA servers, HMIs, historians)	SCADA ↔ PLCs/RTUs, historian feeds, brokered telemetry (MQTT/AMQP)	Conditionally both needed
L3.5 – Industrial DMZ	Jump hosts, patch mgmt, brokers, diodes	Minimum Auth
L4 – Enterprise IT / Edge-to-Cloud	Apps, analytics, MES/ERP, REST/GRPC, MQTT over TLS	Always BOTH
L5 – Business/Cloud	Cloud APIs, dashboards, data lakes	Always BOTH

3.2 Data and Impact Inventory

After assigning assets to the appropriate Purdue layers, a **Data and Impact Inventory** should be created. This inventory captures the operational and security significance of each asset or conduit. Typical information includes data impact level, confidentiality/integrity/availability weights, data lifetime, business impact, safety sensitivity, and whether personally identifiable information is present.

3.3 Cryptographic Inventory

A separate **Cryptographic Inventory** should be developed to capture the cryptographic systems, algorithms, protocols, and protection mechanisms currently deployed across the organization. This includes both hardware and software components, such as encryption methods, authentication mechanisms, certificates, key-management systems, and protocol-level security controls. The inventory may be assembled through manual assessment, automated scanners, PKI or certificate-management review, and network scanning.

4. Dataset Preparation for Upload

4.1 Important formatting rules

For the website upload to work correctly, the dataset should follow these rules:

- Use the exact column names expected by the platform.
- Keep values consistent across all rows.

- Avoid blank entries in critical fields such as Purdue layer, security protocol, device type, exposure, and authentication type.
- Use one row per conduit or communication path.
- Keep categorical values in the accepted format listed in Section 4.3.

4.2 Recommended required columns

The following columns should be included in the uploaded dataset:

`ID`, `Purdue Layer`, `Sec_Protocol`, `Sec_Placement`, `Data Lifetime`, `Device`, `Exposure`, `C_level`, `A_level`, `I_level`, `Business impact`, `Protocols`, `Has_Enc`, `Auth_Type`, `Firmware upgradable`, `Crypto modifiable`, `Vendor Support of PQC`, and `Contains PII`.

Additional descriptive columns such as `Conduit Name`, `Zones`, `Has_Int`, `DATA Type`, and `S_level` can also be included when available.

4.3 Unified PQC and Migration aware conduit inventory (UPMI) attribute guide

In the UPMI, all inventory information and their associated relationships are merged into a single dataframe together with the relevant Purdue layer mappings, creating one unified view for further analysis.

Table 2: UPMI dataset fields and recommended value formats.

Field	Purpose	Recommended values / format
`ID`	Unique identifier for each conduit entry.	Use a unique value per row, for example `SG-01` or `CHEM-12`.
`Conduit Name`	Human-readable name of the conduit.	Free text, for example `Spot Market - Energy Market Subsystem`.
`Zones`	Source and destination zones connected by the conduit.	Free text based on the modeled system, for example `Market - Energy Market Subsystem`.
`Purdue Layer`	Layer span of the communication path.	Use values containing `L0`, `L1`, `L2`, `L3`, `L4`, or `L5`, for example `L2-L2`, `L3-L2`, or `L5-L4`. For the current logic, `L2-L2` should be written exactly when that case applies.
`Protocols`	Main protocol and link context for the conduit.	Include recognizable protocol or network terms such as `Ethernet`, `Wireless`, `VPN`, `TLS`, `WebService`, `Internet`, or combinations such as `Ethernet/TLS`.
`Sec_Protocol`	Security protocol currently protecting the conduit.	Include recognized security protocols such as `TLS`, `IPsec`, `MACsec`, `SSH`, `DLMS`, `COSEM`, `OPC UA`, `DNP3`, `BACnet/SC`, or `HTTPS`. Example: `TLS 1.2/1.3`.
`Has_Enc`	Whether encryption is present.	Use `YES` or `NO`. `Partial` can be retained for documentation, but `YES/NO` is the safest format for platform execution.

Field	Purpose	Recommended values / format
`Auth_Type`	Authentication type used by the conduit.	Use `ServerCert` or `MutCert` for the current recommender logic. Other descriptive values such as `PSK` or `None` may be stored for documentation, but the platform recognizes certificate-based forms most reliably.
`Has_Int`	Whether integrity protection is present.	Recommended values: `YES`, `NO`, or `Partial`.
`Sec_Placement`	Where security is applied.	Recommended values: `EndToEnd`, `GatewayOnly`, `LAN only`, `WAN`, or `local only`. `GatewayOnly` is especially important for constrained deployments.
`DATA Type`	Main data type carried on the conduit.	Free text such as `telemetry`, `control`, `logs`, or `commands`.
`C_level`	Confidentiality level.	Use `H`, `M`, or `L`.
`A_level`	Availability level.	Use `H`, `M`, or `L`.
`I_level`	Integrity level.	Use `H`, `M`, or `L`.
`S_level`	Safety sensitivity level.	Use `H`, `M`, or `L` when included.
`Business impact`	Operational or business impact if the conduit is compromised.	For upload compatibility, `L/M/H` is recommended. If a descriptive scale is used elsewhere, keep it consistent.
`Data Lifetime`	Required protection lifetime.	Recommended values: `10y+`, `5-10y`, `1-5y`, or `<1y`. A `10y` or `10y+` value is interpreted as the highest long-term requirement.
`Contains PII`	Whether personally identifiable information is present.	Use `Yes` or `No`.
`Exposure`	Exposure to untrusted or external networks.	Use consistent values such as `Low`, `Internet`, `WAN`, `LAN/Internal`, or `Isolated`. Where a descriptive label is needed, examples include and every text should be end with degree such high, low, medium such as `Internet / external B2B (High)`.
`Device`	Device class involved in the conduit.	Use a descriptive device name and clearly indicate class, such as `Substation controller (Resource Constrained)`, `HMI server (Servers)`, or `Gateway`. `Resource Constrained`, `Servers`, and `gateway` are important recognized keywords.
`Firmware upgradable`	Whether firmware can be updated.	Use `Yes`, `No`, or `Unknown`.
`Crypto modifiable`	Whether the crypto stack can be changed or replaced.	Use `Yes`, `No`, or `Unknown`.

Field	Purpose	Recommended values / format
`Vendor Support of PQC`	Current vendor support for PQC.	Use `Yes`, `No`, or `Unknown`.

5. How to Use the Platform

5.1 Step 1: Upload the dataset

Prepare the UPMI dataset as a CSV or spreadsheet export and upload it through the MIRAI Shield interface. Each row should represent one conduit. After upload, the platform parses the columns and prepares them for the policy engine.

5.2 Step 2: Run the Policy Engine

Click **Policy Engine** after the dataset upload is complete. The policy engine performs a rule-based evaluation to determine the required protection level and whether deployment is realistically possible on the target communication path.

Main policy output

The key output is **Possible Deployed CAT**. This is the PQC security category that can realistically be deployed for the conduit after balancing security requirement and practical constraints.

Typical outputs are:

- **CAT-1**
- **CAT-3**
- **CAT-5**
- **PQC unprotected** when secure deployment is not currently feasible

5.3 Step 3: Run the Profile Recommender

After the policy result is available, click **Profile Recommender**. The recommender combines the deployed category, benchmark data, device class, protocol context, and latency assumptions to rank candidate KEM and signature algorithms.

Latency and device interpretation

The current implementation maps Purdue-layer context to latency buckets and device constraints as follows:

- **Strict bucket**: layers containing `L0`, `L1`, or `L2`
- **Medium bucket**: layers containing `L3`
- **Loose bucket**: layers containing `L4` or `L5`

The recommender also distinguishes among constrained, embedded, and server-like device classes, which affects the weighting of CPU cost, byte overhead, and RAM usage.

Recommender outputs

For each conduit, the interface reports:

- **Algorithm Rank**
- **Best Algorithm**
- **Confidence Score**
- **Migration Priority**

6. Interpreting the Output

6.1 Algorithm rank

The ranked list orders candidate algorithms from most suitable to least suitable under the selected device and latency conditions. The ranking is utility-based, so lower utility values indicate better overall fit.

6.2 Best algorithm

The best algorithm is the top-ranked candidate among the feasible options for the selected conduit and security category.

6.3 Confidence score

The confidence score indicates how strongly the system favors the top candidate over nearby alternatives. It is derived from the utility ranking and should be interpreted as a decision-support score, not as a direct cryptographic security guarantee.

6.4 Migration priority

Migration priority combines risk urgency, feasibility, and migration friction. In the current design, the priority score is computed from a weighted combination of risk urgency, feasibility, and cost/friction, and then mapped to **High**, **Medium**, or **Low**. In practical terms, a high-priority conduit is both important to protect and realistic to address early in the migration plan.

7. Recommended Review Process

To use MIRAI Shield effectively, practitioners should review outputs in the following order:

1. Confirm that the uploaded dataset uses the correct schema and normalized values.
2. Check the **Possible Deployed CAT** returned by the policy engine.
3. Review the **Best Algorithm** and the top few ranked alternatives.
4. Inspect the **Confidence Score**; lower confidence suggests closer competition among candidates.
5. Use **Migration Priority** to phase the rollout across conduits.
6. Validate the final decision through expert review before operational deployment.

8. Common Upload Mistakes

- Renaming columns or changing capitalization in required headers.
- Writing inconsistent category labels across rows.
- Using free-form authentication labels when the platform expects `ServerCert` or `MutCert`.
- Leaving device class ambiguous instead of clearly marking `Resource Constrained`, `Servers`, or `gateway`.
- Mixing exposure wording without also indicating the intended level clearly.
- Omitting `GatewayOnly` in constrained cases where gateway-based deployment is intended.

8.1 Default Weights and User Customization

This section explains the default scoring equations used by MIRAI Shield and clarifies when an organization may choose to modify the default weights. The platform is designed to work with the default values immediately after upload and policy execution. However, if an organization has specific operational, regulatory, or business requirements, the same weights may be customized through the platform input settings before running the final backend calculation.

Utility score used for profile ranking. For each candidate profile p, MIRAI Shield computes a normalized utility score. Lower utility values indicate a better fit for the conduit under the selected device class, latency bucket, and protocol context.

The general scoring form is

$$U_i(p) = w_1\, z_{\mathrm{CPU}}(x_i, p) + w_2\, z_{\mathrm{BYTES}}(x_i, p) + w_3\, z_{\mathrm{RAM}}(x_i, p) + w_4\, \mathrm{RiskPenalty}(p) + w_5\, \mathrm{InteropPenalty}(p). \tag{1}$$

The normalized terms are defined as

$$z_{\mathrm{CPU}}(p) = \frac{\mathrm{CPU}(p)}{\mathrm{CPU}_{\mathrm{ref}}(\mathrm{device_class})}, \qquad z_{\mathrm{RAM}}(p) = \frac{\mathrm{RAM}(p)}{\mathrm{RAM}_{\mathrm{ref}}(\mathrm{device_class})}, \qquad z_{\mathrm{BYTES}}(p) = \frac{\mathrm{BYTES}(p)}{\mathrm{BYTES}_{\mathrm{ref}}}. \tag{2}$$

In the current version of the framework, $\mathrm{RiskPenalty}(p)$ and $\mathrm{InteropPenalty}(p)$ are not included in the implemented score. Accordingly, the effective utility equation is reduced to CPU, byte overhead, and, where relevant, RAM.

Default utility weights. MIRAI Shield uses different default utility weights for server-class and MCU/embedded-class devices because their practical bottlenecks are different.

For server or gateway settings, the default utility score is

$$U_{\mathrm{server}}(p) = 0.7\, z_{\mathrm{CPU}}(p) + 0.3\, z_{\mathrm{BYTES}}(p). \tag{3}$$

For MCU or embedded settings, the default utility score is

$$U_{\mathrm{mcu}}(p) = 0.6\, z_{\mathrm{CPU}}(p) + 0.2\, z_{\mathrm{BYTES}}(p) + 0.2\, z_{\mathrm{RAM}}(p). \tag{4}$$

The reason for these defaults is practical deployment behavior. On servers and gateways, RAM pressure is typically much less restrictive than computation time and communication overhead during PQC operations, so the RAM weight is set to zero in the default server-side utility calculation. In contrast, on MCU and embedded devices, memory is often a hard constraint, so RAM is explicitly included in the default score.

Ranking rule. Candidate profiles are ranked in ascending order of utility:

$$\text{ranked_profiles}_i = \text{sort}_{p \in P_i}\, U_i(p) \quad \text{(ascending)}. \tag{5}$$

If the selected authentication mode is mutual certificate authentication, the sender- and receiver-side costs are both reflected in the scoring process.

Confidence score. After ranking, MIRAI Shield computes a softmax-style confidence score to indicate how strongly the top-ranked candidate is favored over nearby alternatives. Let

$$U_{\min} = \min_j U_j \tag{6}$$

and define the temperature as

$$\tau = \alpha\, U_{\min}, \tag{7}$$

where α is a tunable strictness parameter. The default value is

$$\alpha = 0.05. \tag{8}$$

Because lower utility is better, the shifted score is computed as

$$S_i = -\frac{U_i - U_{\min}}{\tau}, \tag{9}$$

followed by

$$e_i = e^{S_i}, \qquad P_i = \frac{e_i}{\sum_j e_j}. \tag{10}$$

A higher confidence score means the best-ranked option is more clearly separated from the remaining feasible candidates.

Migration priority score. MIRAI Shield also computes a migration priority score for each conduit by combining risk urgency, feasibility, and cost/friction.

The risk urgency score is

$$R_i = 0.45E_i + 0.35I_i + 0.20L_i, \tag{11}$$

where E_i is the exposure score, I_i is the business impact score, and L_i is the data lifetime score.

The feasibility score is

$$F_i = 0.40FW_i + 0.40CM_i + 0.20V_i, \tag{12}$$

where FW_i is firmware upgradability, CM_i is crypto modifiability, and V_i is vendor PQC support. The cost/friction score is

$$C_i = 0.40S_i + 0.30Pu_i + 0.30D_i, \tag{13}$$

where S_i is security placement friction, Pu_i is Purdue-layer friction, and D_i is device friction. The final migration priority score is

$$\text{PriorityScore}_i = 0.50R_i + 0.30F_i - 0.20C_i. \tag{14}$$

The priority level is then interpreted as follows:

$$\text{PriorityLevel}_i = \begin{cases} \text{High}, & \text{PriorityScore}_i \geq 0.50, \\ \text{Medium}, & 0.30 \leq \text{PriorityScore}_i < 0.50, \\ \text{Low}, & \text{PriorityScore}_i < 0.30. \end{cases} \tag{15}$$

Why these defaults are used. The default coefficients are intended to provide a balanced baseline for practical IIoT migration planning. The utility-score defaults emphasize CPU and communication overhead on server-class systems, while also accounting for RAM on constrained platforms. The migration-priority defaults emphasize risk urgency first, feasibility second, and cost/friction as a reducing factor. This design helps the framework recommend options that are not only secure, but also realistically deployable.

When customization is appropriate. Organizations may keep the default weights when no domain-specific policy is required. However, custom weights may be more appropriate when operational priorities differ from the baseline assumptions. For example, an organization may choose to increase the RAM weight for highly constrained endpoints, increase the exposure or business-impact weight for safety-critical environments, or reduce the cost penalty when rapid migration is strategically required.

Platform behavior. By default, MIRAI Shield executes all calculations using the predefined weights shown above. If a user chooses custom weighting through the platform input settings, the user-provided values are sent to the backend and applied to the same equations. In other words, the equations remain unchanged, while only the coefficients are replaced. This ensures that the scoring logic stays consistent and explainable even when organization-specific tuning is used.

Recommended practice. When custom weights are used, they should be documented as part of the organization's migration policy so that the resulting recommendations remain transparent, reproducible, and reviewable by security, operations, and system engineering stakeholders.

9. Illustrative Case Study: Smart Grid UPMI Example

To demonstrate the practical use of **MIRAI Shield**, an illustrative **Smart Grid case study** can be presented at the end of the manual using a structured **UPMI** dataset. This example helps practitioners understand how a real industrial dataset can be transformed into a unified inventory and then processed by the **Policy Engine** and the **PQC Profile Recommender**.

In this case study, each row of the dataset represents a single **communication conduit**. The dataset includes the main attributes required by MIRAI Shield, such as the conduit identifier, conduit name, zones, Purdue layer, protocols, current security protocol, security placement, data type, C/A/I/S levels, business impact, data lifetime, exposure level, device type, and migration-feasibility attributes. By organizing these attributes into one unified structure, the dataset becomes suitable for direct analysis within the framework.

The Smart Grid example is especially useful because it contains conduit records across multiple **Purdue layers**, ranging from enterprise-level and market-facing communications to operational, substation, and field-level communications. As a result, the case study demonstrates how different communication contexts lead to different policy outcomes, required PQC security categories, and migration priorities.

This example also shows how MIRAI Shield can handle a variety of security conditions. Some conduits may already use protocol-level protection, while others may rely on gateway-based protection, partial protection, or may have no built-in cryptographic protection at all. By processing such diverse records, the framework can illustrate how the policy engine determines the *possible deployed CAT level*, and how the recommender produces a ranked list of candidate algorithms together with a **confidence score**, **best algorithm**, and **migration priority**.

Therefore, this Smart Grid case study serves as a practical reference for users who want to understand how to prepare their own datasets and interpret the final outputs of the framework. It also provides a clear example of how MIRAI Shield can support structured, explainable, and migration-aware PQC planning in real IIoT environments.

9.1 Example Dataset Link

The Smart Grid UPMI example used for demonstration can be provided through the following spreadsheet link:

`https://docs.google.com/spreadsheets/d/1V5GDf1WEGMkfJR2hT2p9wkd1PuCQxXgk66CxyIcPrKM/edit?usp`

10. Conclusion

MIRAI Shield provides a practical path from industrial system modeling to migration-oriented PQC recommendations. By combining Purdue-layer-aware inventories, a rule-based policy engine, and a utility-based profile recommender, the platform helps practitioners transform raw infrastructure data into structured decisions about protection level, deployable algorithms, and migration order. Full algorithm details about this framework is available in full algorithm specification guide section.